

AgentTelemetry: An OpenTelemetry-Native Observability SDK for AI Agent Systems

Krishna Chaitanya Balusu
Independent Researcher
San Francisco, USA
krishnabkc15@gmail.com

Abstract—LLM-based AI agents fail in modes—circular delegation, planning loops, silent guardrail bypass, stale memory—that existing observability tools cannot represent. OpenTelemetry’s GenAI semantic conventions cover LLM invocation and tool execution but leave five agent orchestration phases (planning, reasoning, safety, delegation, memory) without span-level vocabulary, so the resulting traces show *what* the model said but not *what the agent did*. We present AGENTTELEMETRY, an Apache-licensed, pip-installable Python SDK that adds nine agent-specific span kinds, seven framework auto-instrumentors (LangChain, CrewAI, AutoGen, OpenAI, Anthropic, LlamaIndex, plus a manual API), three privacy levels, and four analysis modules (anomaly detection, cost aggregation, decision attribution, hallucination tracing). The SDK is OpenTelemetry-native and exports to any OTLP backend (Jaeger, Grafana Tempo, Datadog, etc.). A developer instruments an existing LangChain agent in five lines and immediately gains structural visibility into coordination failures. A companion 3,780-row public benchmark (Zenodo DOI 10.5281/zenodo.20129005) shows that the nine-kind span vocabulary can express fault classes that vanilla OpenTelemetry cannot. All artifacts—PyPI release, source, dataset, and a 3–5 minute screencast—are openly available.

Index Terms—AI agents; observability; OpenTelemetry; fault detection; distributed tracing; LLM monitoring; software tools

I. INTRODUCTION

Autonomous LLM agents now plan tasks, call tools, delegate to sub-agents, retrieve from vector stores, and reason over multi-turn state. When they fail, however, the trace a developer sees in their observability backend rarely reflects the agent’s actual cognitive trajectory. OpenTelemetry’s GenAI semantic conventions [1], [2] model only LLM invocation and tool execution. Failures that originate from *coordination* (circular delegation, agent misroute, infinite reasoning loops) or from *cognitive state* (stale retrieval, memory corruption, guardrail bypass) [3], [4] therefore appear as indistinguishable nests of `gen_ai.completion` spans.

Existing commercial and open-source agent observability tools—LangSmith [5], Langfuse [6], AgentOps [7], Arize Phoenix [8], OpenLIT [9]—either reuse the generic GenAI spans or introduce closed proprietary schemas ([10] partially excepted). None offers a typed span vocabulary for the five missing orchestration phases, and several are SaaS-only.

AGENTTELEMETRY closes this gap with an OpenTelemetry-native SDK. A developer who already understands OTEL does not need to learn a new dashboarding product or accept a hosted backend; they `pip install`

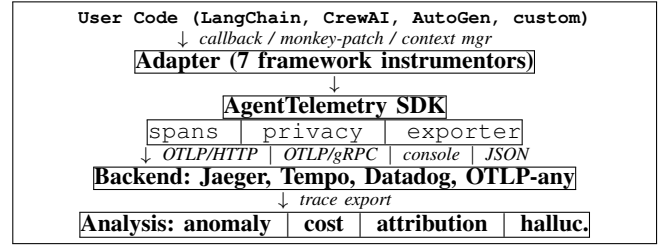


Fig. 1. AgentTelemetry architecture. User code is instrumented through a framework adapter or the manual API. Spans flow through the SDK’s privacy filter and exporter to any OTLP-compatible backend. Analysis modules consume completed traces offline to flag structural failure patterns.

`agenttelemetry`, add two lines, and their existing Jaeger or Tempo instance starts showing typed spans for `PLANNING`, `REASONING`, `DELEGATION`, `GUARD_RAIL`, and `MEMORY`. The same span graph then feeds four offline analysis modules that flag known agent failure patterns structurally, without inspecting prompt content.

Contributions. This tool demonstration paper contributes (i) a pip-installable, Apache-2.0 SDK (4,100 LOC, 111 tests) implementing nine agent span kinds and seven framework adapters; (ii) four analysis modules that operate over the resulting traces; (iii) a publicly downloadable 3,780-row fault detection benchmark on Zenodo that demonstrates the SDK produces a detectable signal across 14 fault classes; and (iv) a 3–5 minute screencast walking through end-to-end instrumentation, trace inspection in Jaeger, and anomaly detection.

II. TOOL OVERVIEW

Span kinds. The SDK defines an `AgentSpanKind` enum with nine values: `AGENT`, `LLM_CALL`, `TOOL_CALL`, `PLANNING`, `REASONING`, `RETRIEVAL`, `GUARD_RAIL`, `DELEGATION`, and `MEMORY`. Each is exported as the standard `agent.span.kind` OTEL attribute, so existing OTLP backends display them without modification. Three of the nine (`LLM_CALL`, `TOOL_CALL`, `RETRIEVAL`) overlap conceptually with OTEL GenAI; the remaining six fill the orchestration gap.

Adapters. Table I summarises the seven framework adapters. Each adapter uses lazy imports: the framework package is required only at `instrument()` time, so users do not pay an import cost for frameworks they don’t use.

TABLE I
FRAMEWORK ADAPTERS SHIPPED WITH AGENTTELEMETRY 0.1.0.

Framework	Class	Strategy	LOC
LangChain	LangChainInstr.	Callback handler	359
CrewAI	CrewAIInstr.	Hook system	264
AutoGen	AutoGenInstr.	Monkey-patch	256
Anthropic SDK	AnthropicInstr.	Monkey-patch	294
OpenAI SDK	OpenAIInstr.	Monkey-patch	245
LlamaIndex	LlamaIndexInstr.	Span handler	280
Custom (manual)	start_agent_span	Context mgr	315

Privacy. Three levels (NONE, METADATA_ONLY [default], FULL) gate what attribute payloads reach the exporter; metadata-only retains structural diagnostics (model names, token counts, USD cost, status) while suppressing prompt and completion text.

Analysis modules. Four modules consume completed traces offline: AnomalyDetector flags circular delegation, infinite retries, cost explosions, and context overflow by walking the span graph; CostAggregator sums per-model and per-agent USD spend; DecisionAttributor links each tool call back to the LLM decision that triggered it; HallucinationTracer marks LLM outputs whose tokens are not grounded in spans of kind RETRIEVAL immediately preceding them. All four operate on serialized trace JSON, so they can run on Jaeger exports, persisted .jsonl bundles, or live OTel collector pipelines.

III. USAGE

Listing 1 shows the entire developer-facing surface for a hand-instrumented agent: one configure call and a nested set of context managers. The same trace can be exported to Jaeger by setting OTEL_EXPORTER_OTLP_ENDPOINT; no SDK-side change is required.

Listing 1. Manual instrumentation in 10 lines.

```
from agenttelemetry import (
    configure, start_agent_span, AgentSpanKind)

provider = configure(service_name="my-agent", console=True)
tracer = provider.get_tracer()

with start_agent_span("research", AgentSpanKind.AGENT,
    tracer=tracer):
    with start_agent_span("call-gpt4", AgentSpanKind.LLM_CALL,
        tracer=tracer) as llm:
        llm.set_attribute("llm.model", "gpt-4o")
        llm.set_attribute("llm.input_tokens", 500)
    with start_agent_span("search", AgentSpanKind.TOOL_CALL,
        tracer=tracer) as tool:
        tool.set_attribute("tool.name", "google")
```

Listing 2 shows the equivalent for an existing LangChain [11] ReAct [12] agent. The LangChainInstrumentor attaches a callback handler that maps LangChain’s internal events (chain start, LLM start, tool start, agent action) onto the correct AgentSpanKind, so *no application code changes*.

Listing 2. Auto-instrumentation for an existing LangChain agent.

```
from agenttelemetry import configure
from agenttelemetry.adapters import (
```

```
LangChainInstrumentor)
```

```
provider = configure(service_name="my-agent")
LangChainInstrumentor().instrument(
    tracer_provider=provider.tracer_provider)
# existing LangChain code runs unchanged
```

A trace produced by Listing 2 on a tool-using ReAct agent appears in Jaeger as the nested span graph AGENT → PLANNING → LLM_CALL → TOOL_CALL → REASONING → LLM_CALL, with the orchestration spans appearing alongside the standard GenAI spans rather than replacing them. This is the key visual payoff of the SDK and is the centerpiece of the accompanying screencast (Sec. VI).

Walk-through: catching a circular delegation. Consider a CrewAI swarm (examples/multi_agent.py in the repository) in which agent A delegates a sub-task to agent B, which routes back to A on an unrecognized intent. Without typed spans the resulting trace is a deep nest of LLM completions and tool calls—no single span attribute signals the cycle. Auto-instrumented with CrewAIInstrumentor, the same run produces the span graph shown in Listing 3: every cross-agent hand-off emits a DELEGATION span carrying agent.delegation.target and agent.delegation.depth attributes. The bundled AnomalyDetector traverses this graph with three lines of code and returns a list of typed Anomaly records (Listing 3, bottom). The same call also flags cost explosions and infinite retries from the same trace. This end-to-end loop—instrument, export, analyse—runs in under 30 seconds on a developer laptop and is the centre of the demo video.

Listing 3. Jaeger-style trace dump and AnomalyDetector output for the circular-delegation case.

```
AGENT researcher [span-a1]
PLANNING decide-route [span-a2]
  LLM_CALL gpt-4o [span-a3]
    DELEGATION -> writer [span-a4] target=writer depth=1
      AGENT writer [span-b1]
        DELEGATION -> researcher [span-b2] target=researcher
          depth=2
            AGENT researcher [span-a1] <-- cycle

>>> AnomalyDetector().detect(trace)
[Anomaly (type='CIRCULAR_DELEGATION',
    evidence=['span-a1', 'span-b2', 'span-a1'],
    depth=3, severity='high')]
```

IV. COMPARISON WITH EXISTING TOOLS

Table II contrasts AgentTelemetry with the five most prominent agent-observability tools as of mid-2026. The per-competitor wedge is the load-bearing claim of this section: against **LangSmith** [5] (closed-source SaaS, no OTLP egress), AgentTelemetry is self-hostable Apache code that exports to any OTLP backend the team already runs. Against **Langfuse** [6] (OTel-native but generic LLM spans), AgentTelemetry adds five typed orchestration span kinds (PLANNING, REASONING, DELEGATION, GUARD_RAIL, MEMORY) plus an AGENT root span that Langfuse currently folds into untyped trace events. Against **AgentOps** [7] (OSS dashboard product), AgentTelemetry ships an offline analysis library (AnomalyDetector, CostAggregator, DecisionAttributor, HallucinationTracer) that operates on exported trace JSON without a

hosted backend. Against **Phoenix** [8] (OpenInference-based), AgentTelemetry stays on the OpenTelemetry semantic-conventions track and contributes a draft OTEP (otep/agent-semantic-conventions.md) rather than maintaining a parallel schema. Against the closest competitor, **OpenLIT** [9] (Apache, OTel-native, 20+ provider instrumentations), the wedge is structural: OpenLIT instruments individual *provider SDKs* (Anthropic, OpenAI, vector stores) and emits standard `gen_ai.*` spans, so a CrewAI delegation cycle is invisible at the span-kind level. AgentTelemetry instruments *agent frameworks* (LangChain, CrewAI, AutoGen) and emits typed orchestration spans, so the same cycle shows up directly as a `DELEGATION` subgraph that `AnomalyDetector` can match without parsing prompt text.

V. EVIDENCE OF DETECTABLE SIGNAL

A 3,780-row controlled benchmark accompanies the SDK release (3,528 fault-injection runs + 252 fault-free controls) [13], [14] and is publicly downloadable as a tab-separated file from the Zenodo record. The benchmark covers 14 fault classes across six telemetry conditions and seven agent frameworks. Aggregate Fault Detection Rate (FDR), reproducible verbatim from DOI 10.5281/zenodo.20129006:

Telemetry condition	FDR	Detected
No telemetry	0.000	0/14
Vanilla OpenTelemetry	0.429	6/14
OTel GenAI conventions	0.429	6/14
OpenInference	0.429	6/14
AgentTelemetry (metadata privacy)	0.612	7/14
AgentTelemetry (full vocabulary, upper bd.)	1.000	14/14

The upper bound is the relevant claim for a tool-demonstration audience: the nine-kind span vocabulary is structurally *sufficient* to express all 14 fault classes, including the eight coordination, cognitive, and policy faults (`circular_delegation`, `hallucination`, `stale_retrieval`, `guardrail_bypass`, `planning_failure`, `reasoning_loop`, `agent_misroute`, `memory_corruption`) that no baseline can express. The 0.612 mid-row is a per-framework conformance gap, not a metamodel limit: only the bundled `custom_agent` reference adapter currently emits the full nine-kind vocabulary, while third-party adapters emit 3–7 typed kinds out of 9. Closing this gap is ongoing work and is tracked in the repository roadmap. Full methodology, ablations, and statistical analysis are reported separately [13]; we reference the artifact here strictly as evidence that the SDK produces a usable signal.

Overhead. The in-tree benchmark `tests/benchmarks/test_overhead.py` (run against an `InMemorySpanExporter`) asserts that a single instrumented span completes in <5 ms and a nested agent→LLM→tool triple in <10 ms on a 2024-class laptop CPU, well below typical LLM round-trip latencies.

VI. AVAILABILITY AND REPRODUCIBILITY

Install. `pip install agenttelemetry==0.1.0` (PyPI, Apache-2.0) [15]. The unpinned `pip install`

`agenttelemetry` also resolves to 0.1.0 today. On a stock 2024-class laptop the install completes in <30 seconds (the SDK has four runtime dependencies: `opentelemetry-api`, `opentelemetry-sdk`, `opentelemetry-exporter-otlp`, `pydantic`). Optional extras (`[langchain]`, `[crewai]`, `[autogen]`, `[anthropic]`, `[openai]`, `[llamaindex]`, `[all]`) pull in the corresponding framework dependency groups.

Source. <https://github.com/Krishnachaitanyak/AgentTelemetry> (Apache-2.0, 4,100 SDK LOC, 111 tests, CI on Python 3.9–3.12).

Archived artifact. Zenodo concept DOI [10.5281/zenodo.20129005](https://doi.org/10.5281/zenodo.20129005); pinned version DOI [10.5281/zenodo.20129006](https://doi.org/10.5281/zenodo.20129006) contains the 0.1.0 source plus the 3,780-row benchmark TSV and a `requirements.lock` that reproduces every number above verbatim.

Screencast. A 3–5 minute video walks through installation, manual and auto-instrumentation, the Jaeger trace view, and the anomaly detector on a circular-delegation example: <https://youtu.be/AGENTTELEMETRY-ICSE27-DEMO>.

Semantic conventions. A draft OTEP proposing the nine agent span kinds for upstream adoption lives at `otep/agent-semantic-conventions.md` in the repository and is being discussed with the OpenTelemetry Semantic Conventions working group.

VII. LIMITATIONS AND ROADMAP

The SDK is at **alpha** maturity (PyPI development status 3). Three limitations bound current use. First, the 0.612 average FDR in Sec. V reflects that only the bundled `custom_agent` reference adapter emits the full nine-kind vocabulary; third-party framework adapters emit 3–7 of the nine typed kinds. Conformance work for LangChain, CrewAI, and AutoGen adapters is the top roadmap item. Second, the `HallucinationTracer` relies on a token-overlap heuristic against `RETRIEVAL`-kind spans and is therefore recall-biased; semantic-similarity grounding is a planned replacement. Third, the OTEP that proposes the nine span kinds for upstream OpenTelemetry adoption is under community review and not yet a ratified standard, so backends that match on `agent.span.kind` are configuring against a draft attribute.

VIII. CONCLUSION

AGENTTELEMETRY ships today as a one-command-installable Python SDK that gives any OpenTelemetry-equipped team typed visibility into the orchestration phases that distinguish AI agents from single LLM calls. The accompanying public benchmark and screencast let reviewers and practitioners verify, in minutes, that the span vocabulary captures coordination and cognitive failure modes other tools elide. We invite the community to install the package, build new framework adapters, file fault patterns, and—via the OTEP draft—help shape an upstream OpenTelemetry semantic convention for agent observability.

TABLE II

AGENTTELEMETRY VS. EXISTING AGENT-OBSERVABILITY TOOLS (VERIFIED FROM EACH PROJECT’S OFFICIAL DOCUMENTATION, MAY 2026). ✓ = YES, ✗ = NO.

Tool	OSS	OTel native	Typed agent spans	Adapters	Analysis	Open semconv proposal	Self-host
LangSmith [5]	✗ (SaaS)	✗	generic LLM/tool	many	proprietary	✗	✗
Langfuse [6]	✓	✓	generic LLM	50+	dashboards	✗	✓
AgentOps [7]	✓	not docu.	generic	many	dashboards	✗	✓
Phoenix [8]	✓	via OpenInf.	generic LLM/tool	many	evaluation	✗ (OpenInf., not OTel)	✓
OpenLIT [9]	✓	✓	generic LLM	20+	dashboards	✗	✓
AgentTelemetry (this work)	✓ (Apache)	✓	9 typed kinds	7	4 modules	✓ (OTEP draft)	✓

REFERENCES

- [1] OpenTelemetry Authors, “OpenTelemetry specification v1.30,” 2024. [Online]. Available: <https://opentelemetry.io/docs/specs/otel/>
- [2] —, “Semantic conventions for generative AI systems,” 2024. [Online]. Available: <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
- [3] M. Cemri, M. Z. Pan, S. Yang *et al.*, “Why do multi-agent LLM systems fail?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [4] L. Dong, Q. Lu, and L. Zhu, “AgentOps: Enabling observability of LLM agents,” 2024.
- [5] LangChain Inc., “LangSmith: A framework-agnostic platform for LLM applications,” 2026. [Online]. Available: <https://docs.langchain.com/langsmith>
- [6] Langfuse GmbH, “Langfuse: Open-source LLM engineering platform,” 2026. [Online]. Available: <https://langfuse.com/>
- [7] AgentOps, Inc., “AgentOps: Observability for AI agents and LLM apps,” 2026. [Online]. Available: <https://docs.agentops.ai/>
- [8] Arize AI, “Arize Phoenix: Open-source AI observability,” 2026. [Online]. Available: <https://arize.com/docs/phoenix>
- [9] OpenLIT Authors, “OpenLIT: OpenTelemetry-native AI observability,” 2026. [Online]. Available: <https://openlit.io/>
- [10] Arize AI, “OpenInference: Semantic conventions and instrumentations for AI applications,” 2024. [Online]. Available: <https://github.com/Arize-ai/openinference>
- [11] LangChain, Inc., “LangChain: Building context-aware reasoning applications,” 2024. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [13] K. C. Balusu, “AgentTelemetry: A fault detection benchmark and toolkit for LLM agent observability,” in *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware ’26)*, 2026. [Online]. Available: <https://doi.org/10.1145/3805760.3814931>
- [14] —, “AgentTelemetry: A fault detection benchmark and toolkit for LLM agent observability,” 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.20129006>
- [15] —, “AgentTelemetry 0.1.0 on PyPI,” 2026. [Online]. Available: <https://pypi.org/project/agenttelemetry/>